

MICROVISION

Service Management System

Technical Build Plan v1.1 — Every File, Every Layer, Every Phase

MERN Stack · Railway + Vercel + MongoDB Atlas + Cloudinary + Brevo + Shadcn/ui

1. Production Stack & Why Each Tool

Every tool chosen works in production on Railway Hobby (\$5/month) and Vercel (free). No exceptions.

Layer	Tool	Why This One	Cost
Backend hosting	Railway Hobby	Always-on, GitHub deploy, no sleep, \$5 flat, 100GB ephemeral disk	\$5/month
Frontend hosting	Vercel	Best for React/Vite, free, global CDN, auto-deploy on every push	Free
Database	MongoDB Atlas M0	Free 512MB — enough for years of complaint + billing data	Free
Image/file storage	Cloudinary	25GB free, auto-compress on upload via transformation config, CDN delivery	Free (1-2yr+)
Email — OTP + notifications	Brevo HTTP API	Railway Hobby blocks SMTP entirely. Brevo uses HTTPS API — never blocked. 300 emails/day free forever.	Free
WhatsApp notifications	WATI or AiSensy	Indian providers, cheaper than Twilio, simple HTTP API, WhatsApp template support	Per message
UI component library	Shadcn/ui + Tailwind	Copy-paste components, fully customizable, no bundle bloat — only imports what you use. Saves 40-50% frontend time.	Free
PWA	vite-plugin-pwa	Zero config, auto service worker, auto-updates on every deploy	Free

⚠ Critical: SMTP is blocked on Railway Hobby — confirmed from official pricing page

- Railway Hobby and Free plans have SMTP disabled. Pro (\$20/month) and Enterprise only have it.
- Nodemailer + Gmail works locally because your home internet allows SMTP. Railway's servers do not.
- Fix: Use Brevo's HTTP API. One POST request with API key. Railway never blocks HTTPS.
- Brevo free = 300 emails/day. Your app needs ~10-20/day max. Completely fine, forever free.
- Code: a simple `sendEmail(to, subject, html)` util using axios to `api.brevo.com` — 15 lines total.

2. Shadcn/ui — What It Is & How to Use It

Shadcn/ui is not a traditional component library. It copies component source code directly into your project — you own the code, customize it freely, and only ship what you use. Built on Radix UI primitives + Tailwind CSS.

2.1 Setup — one command

```
npx shadcn@latest init
```

This sets up Tailwind, creates a components/ui folder, and configures everything. Then add individual components as needed:

```
npx shadcn@latest add table
npx shadcn@latest add dialog
npx shadcn@latest add select
npx shadcn@latest add tabs
npx shadcn@latest add badge
npx shadcn@latest add form input textarea
npx shadcn@latest add toast
npx shadcn@latest add card
npx shadcn@latest add sheet
npx shadcn@latest add dropdown-menu
npx shadcn@latest add avatar
npx shadcn@latest add separator
npx shadcn@latest add skeleton
npx shadcn@latest add alert
```

2.2 Shadcn components mapped to your features

Your Feature	Shadcn Component Used	What it replaces
Complaint list, billing list	Table + TableHeader + TableRow + TableCell	Writing full table HTML/CSS from scratch
Approve/reject modals, reopen modal, SC picker	Dialog + DialogContent + DialogHeader + DialogFooter	Custom modal with backdrop, z-index, escape key handling
City dropdown, status filter, preset selector, product type	Select + SelectContent + SelectItem	Custom dropdown with keyboard nav
Admin 4 tabs, SC 3 tabs	Tabs + TabsList + TabsTrigger + TabsContent	Manual tab state + conditional rendering
Status labels (pending, done, rejected)	Badge with variant prop	Custom span with color classes
All forms — complaint, login, register, reset	Form + FormField + FormItem + FormLabel + FormMessage	Manual validation display + field wiring
All text inputs	Input	Plain HTML input with Tailwind classes
Notes, voice note label	Textarea	Plain HTML textarea
Success/error notifications	Toast + Toaster + useToast hook	Custom toast system

Your Feature	Shadcn Component Used	What it replaces
SC cards, complaint cards, stat cards	Card + CardHeader + CardContent + CardFooter	Custom div layouts
Mobile sidebar	Sheet + SheetContent + SheetTrigger	Custom slide-in drawer
Action menu on complaint row	DropdownMenu + DropdownMenuItem	Custom popup menu
Loading skeletons	Skeleton	Custom shimmer animations
Warning / info messages	Alert + AlertDescription	Custom styled divs
User avatar in navbar	Avatar + AvatarFallback	Custom initials circle

2.3 What you still write yourself (not in shadcn)

- VoiceRecorder.jsx — MediaRecorder API logic, timer, upload
- ImageUploader.jsx — drag+drop, preview thumbnails, Cloudinary upload call
- StatusTimeline.jsx — vertical audit log timeline UI
- PetrolEditField.jsx — edit count logic, lock display, 3-round history
- ReopenBanner.jsx — reopen eligibility banner + modal
- ComplaintFilters.jsx — filter bar combining multiple shadcn Selects + Input
- All page-level layout and data fetching logic

These are business-logic-specific components. Shadcn gives you all the generic UI primitives — you wire them together for your app's specific needs.

3. Complete Project Structure

Two repos — backend on Railway, frontend on Vercel. Both auto-deploy from GitHub on every push.

3.1 Backend Repo — microvison-backend

```
microvison-backend/
├── package.json
├── .env                # all secrets — never commit
├── .env.example        # key names only — safe to commit
├── .gitignore
├── server.js           # Express app entry — CORS, routes, DB connect,
listen
├── config/
│   ├── db.js           # mongoose.connect() with error handling
│   ├── cloudinary.js   # cloudinary.config() from env vars
│   └── brevo.js         # Brevo base URL + API key + sendEmail() helper
├── models/
│   ├── User.js         # name, email, passwordHash, role, status
│   ├── ServiceCentre.js # userId ref, businessName, phones, emails, city,
district, state, productCapability, status
│   ├── Complaint.js    # full complaint schema — all fields from GRD
│   └── ComplaintUpdate.js # audit log — every status change with timestamp +
actor
│   ├── Preset.js       # type, name, price, isActive
│   ├── City.js         # name, district, state — seeded once
│   └── Invoice.js       # serviceCentreId, month, year, totalAmount,
complaintRefs
│   └── OtpToken.js     # email, code, expiresAt — TTL auto-deletes
├── middleware/
│   └── auth.js          # verifyToken — extract + verify JWT, attach
req.user
│   ├── rbac.js         # isAdmin / isSC / isActive guards
│   └── upload.js       # multer + multer-storage-cloudinary, image
compress config
│   └── errorHandler.js # global Express error handler
├── routes/
│   ├── auth.routes.js  # login, register, forgot, verify-otp, reset
│   ├── user.routes.js  # get profile, update password
│   ├── serviceCentre.routes.js # CRUD + approve/reject/deactivate
│   ├── complaint.routes.js # create, assign, status update, reopen, filters
│   ├── preset.routes.js # CRUD + toggle active
│   ├── city.routes.js  # get all, get by district
│   ├── billing.routes.js # complaint bills, monthly invoice
│   ├── upload.routes.js # image upload, audio upload
│   └── whatsapp.routes.js # internal trigger endpoints
├── controllers/
│   ├── auth.controller.js
│   └── user.controller.js
```

```

|   |— serviceCentre.controller.js
|   |— complaint.controller.js
|   |— preset.controller.js
|   |— city.controller.js
|   |— billing.controller.js
|   |— upload.controller.js
|   |— whatsapp.controller.js
|— utils/
|   |— generateComplaintId.js      # MV-YYYY-XXXXX auto-increment
|   |— sendEmail.js              # Brevo HTTP API wrapper – accepts {to, subject,
html}
|   |— sendWhatsApp.js           # WATI/AiSensy HTTP API wrapper
|   |— billingCalculator.js      # in-warranty vs out-of-warranty bill logic
|   |— reopenChecker.js         # check if complaint eligible for reopen

```

Backend total: 37 files

3.2 Frontend Repo — microvison-frontend

```

microvison-frontend/
|— package.json
|— vite.config.js                # Vite + vite-plugin-pwa config
|— tailwind.config.js           # Tailwind + shadcn CSS variables
|— postcss.config.js
|— components.json              # shadcn config – auto-generated by init
|— .env                         # VITE_API_URL=https://your-railway-url
|— .gitignore
|
|— public/
|   |— favicon.ico
|   |— icon-192.png             # PWA icon 192x192
|   |— icon-512.png            # PWA icon 512x512
|
|— src/
|   |— main.jsx                 # React entry
|   |— App.jsx                  # React Router + ProtectedRoute + role-based
redirect
|   |— api/
|       |— axios.js             # axios instance – baseURL + auto-attach JWT
header
|   |— context/
|       |— AuthContext.jsx      # user state, login(), logout(), token management
|
|   |— hooks/
|       |— useAuth.js           # read current user from AuthContext
|       |— useComplaints.js     # fetch + filter + paginate complaints
|       |— useServiceCentres.js # fetch + filter SCs
|       |— useBilling.js        # fetch billing + invoice data
|
|   |— utils/
|       |— formatDate.js        # DD MMM YYYY, relative time

```

	└─ formatCurrency.js	# ₹ Indian rupee formatter
	└─ statusColors.js	# status string → Badge variant/color
	└─ constants.js	# PRODUCT_TYPES, STATUS_LIST, CAPABILITY_OPTIONS
	└─ components/	
manually	└─ ui/	# shadcn components – auto-generated, DO NOT edit
	└─ button.jsx	# shadcn add button
	└─ input.jsx	# shadcn add input
	└─ textarea.jsx	# shadcn add textarea
	└─ select.jsx	# shadcn add select
	└─ table.jsx	# shadcn add table
	└─ dialog.jsx	# shadcn add dialog
	└─ tabs.jsx	# shadcn add tabs
	└─ badge.jsx	# shadcn add badge
	└─ card.jsx	# shadcn add card
	└─ form.jsx	# shadcn add form
	└─ toast.jsx	# shadcn add toast
	└─ toaster.jsx	# shadcn add toast
	└─ sheet.jsx	# shadcn add sheet
	└─ dropdown-menu.jsx	# shadcn add dropdown-menu
	└─ skeleton.jsx	# shadcn add skeleton
	└─ alert.jsx	# shadcn add alert
	└─ avatar.jsx	# shadcn add avatar
	└─ separator.jsx	# shadcn add separator
	└─ label.jsx	# shadcn add label
	└─ layout/	
	└─ Navbar.jsx	# top bar – logo, user info, logout
custom nav	└─ AdminSidebar.jsx	# 4 tabs + badge counts – uses shadcn Tabs or
	└─ SCSidebar.jsx	# 3 tabs – uses same pattern
	└─ complaint/	
	└─ ComplaintCard.jsx	# shadcn Card – used in list views
	└─ ComplaintDetail.jsx	# full detail – role-aware actions
	└─ StatusTimeline.jsx	# custom vertical timeline (no shadcn equiv)
	└─ BillSummary.jsx	# shadcn Card + Table for bill breakdown
approve/reject	└─ ExtraChargesList.jsx	# shadcn Table + Badge + Dialog for
	└─ PetrolEditField.jsx	# custom – edit count logic + lock state
	└─ ReopenBanner.jsx	# shadcn Alert + Dialog for reopen flow
	└─ forms/	
dropdown)	└─ Step1CustomerInfo.jsx	# shadcn Form + Input + Select (city
	└─ Step2ProductType.jsx	# shadcn Form + radio group pattern
uploaders	└─ Step3Charges.jsx	# PresetSelector + Input + Textarea +
	└─ Step4AssignSC.jsx	# shadcn Card list of SCs + select one
equiv)	└─ VoiceRecorder.jsx	# custom – MediaRecorder API (no shadcn
	└─ ImageUploader.jsx	# custom – drag+drop + preview + Cloudinary
	└─ PresetSelector.jsx	# shadcn Select filtered by product+type

	filters/	
	└─ ComplaintFilters.jsx	# row of shadcn Selects + Input + DatePicker
	└─ SCFilters.jsx	# shadcn Selects + Input for SC list
	└─ BillingFilters.jsx	# shadcn Selects for month/year/SC
	└─ billing/	
	└─ BillingTable.jsx	# shadcn Table – complaint bills
	└─ MonthlyInvoice.jsx	# shadcn Card + Table – monthly rollup
	└─ pages/	
	└─ auth/	
	└─ Login.jsx	# shadcn Form + Input + Button
	└─ Register.jsx	# shadcn Form – full SC registration
	└─ ForgotPassword.jsx	# shadcn Form + Input
	└─ VerifyOtp.jsx	# shadcn Input (6 digit) + countdown
	└─ ResetPassword.jsx	# shadcn Form + Input x2
	└─ admin/	
	└─ AdminLayout.jsx	# AdminSidebar + Navbar + page outlet
	└─ ActionCentre.jsx	# 4 sections with shadcn Card + Badge +
Dialog	└─ ServiceCentres.jsx	# shadcn Table or Card grid + SCFilters
	└─ SCDetail.jsx	# shadcn Card + Tabs (info / complaints)
	└─ AllComplaints.jsx	# shadcn Table + ComplaintFilters +
pagination	└─ Billing.jsx	# shadcn Tabs (per-complaint / monthly) +
filters	└─ Presets.jsx	# shadcn Table + Dialog for add/edit +
AlertDialog	└─ NewComplaint.jsx	# 4-step wizard – step indicator +
conditional	└─ step render	
	└─ sc/	
	└─ SCLayout.jsx	# SCSidebar + Navbar + outlet
	└─ NewRequests.jsx	# shadcn Card list + Accept/Reject Dialog
	└─ MyComplaints.jsx	# shadcn Table + filters + ComplaintDetail
	└─ SCBilling.jsx	# shadcn Card + Table + month filter

Frontend total: 62 files (including 19 shadcn ui files auto-generated)

Custom-written frontend files: 43

Backend files: 37

Grand total: 99 files | ~8,500 lines of hand-written code (shadcn files not counted)

4. Build Phases — Exact Sequence

13 phases. Follow this order strictly — each phase depends on the previous.

Phase 1 — Project Setup & Config (Day 1)

Goal: both repos running locally, DB connected, shadcn initialized, env vars set.

#	File	What to do
1	microvison-backend/package.json	npm init. Install: express mongoose dotenv cors bcryptjs jsonwebtoken multer cloudinary multer-storage-cloudinary axios
2	.env + .env.example	Add all keys: MONGO_URI, JWT_SECRET, CLOUDINARY_*, BREVO_API_KEY, BREVO_FROM_EMAIL, WATI_API_KEY, WATI_BASE_URL, FRONTEND_URL, PORT=5000
3	config/db.js	mongoose.connect(MONGO_URI) — log success or throw on error
4	config/cloudinary.js	cloudinary.config() with cloud_name, api_key, api_secret from env
5	config/brevo.js	Export BREVO_KEY and BASE_URL. Export sendEmail(to, subject, html) using axios POST to api.brevo.com/v3/smtp/email
6	server.js	Express setup: cors({origin: FRONTEND_URL}), express.json(), mount all route files (stubs ok for now), errorHandler, listen on PORT
7	microvison-frontend/ (init)	npm create vite@latest microvison-frontend -- --template react. cd in, npm install.
8	tailwind + shadcn	Install Tailwind. Run: npx shadcn@latest init — choose default style, slate base color, CSS variables yes.
9	shadcn components (all at once)	Run: npx shadcn@latest add button input textarea select table dialog tabs badge card form toast toaster sheet dropdown-menu skeleton alert avatar separator label
10	src/api/axios.js	Create axios instance: baseURL = import.meta.env.VITE_API_URL. Request interceptor: attach Authorization: Bearer <token from localStorage>.
11	vite.config.js	Add vite-plugin-pwa with registerType: autoUpdate, manifest (name: Microvison, icons), workbox config.

Test checkpoint: GET /health returns 200. Frontend loads on localhost:5173 with shadcn Button rendering correctly.

Phase 2 — Database Models (Day 1-2)

Goal: all 8 Mongoose schemas defined. Cities seeded. No routes yet.

#	File	Key fields
1	models/City.js	name (String), district (String), state (String). After creating, run seed script for all Rajasthan + Punjab cities.
2	models/User.js	name, email (unique), passwordHash, role (enum: admin/service_centre), status (enum: active/pending/rejected), createdAt
3	models/ServiceCentre.js	userId (ref User), ownerName, businessName, phone1, phone2, email1, email2, fullAddress, city, district, state, productCapability (enum: led_only/cooler_only/both), status (active/pending/rejected)
4	models/Preset.js	type (enum: installation_led/complaint_led/complaint_cooler), name, price (Number), isActive (Boolean, default true)
5	models/OtpToken.js	email, code (String 6-digit), expiresAt (Date). Add index: {expiresAt: 1, expireAfterSeconds: 0} for auto-delete.
6	models/Complaint.js	All fields from GRD — complaintId, customer fields, product, complaintType, warrantyStatus, presetId, presetPrice, petrol fields x3, petrolEditCount, petrolLocked, extraCharges[], customerPaymentAmount, notes, voiceNoteUrl, adminPhotos[], proofPhotos[], assignedCentreId, status, isReopened, reopenParentId, reopenNotes, reopenPhotos[], billGenerated, whatsappSent, createdBy
7	models/ComplaintUpdate.js	complaintId (ref), updatedBy (ref User), role (String), oldStatus, newStatus, note (String), timestamp (Date, default now)
8	models/Invoice.js	serviceCentreId (ref), month (Number 1-12), year (Number), totalAmount, complaintCount, complaints ([ref Complaint])

Create `utils/seedCities.js` — a one-time script with an array of all cities. Run: `node utils/seedCities.js`. Verify count in Atlas.

Phase 3 — Auth System (Day 2-3)

Goal: login, register, OTP reset working. JWT middleware protecting routes. Brevo emails firing.

#	File	What to build
1	utils/sendEmail.js	Brevo HTTP wrapper: <code>axios.post('https://api.brevo.com/v3/smtp/email', {sender:{name,email}, to:[{email:to}], subject, htmlContent: html}, {headers:{'api-key': BREVO_KEY}})</code>
2	controllers/auth.controller.js — login	Find user by email. If not found or status=pending/rejected → error. <code>bcrypt.compare(password, hash)</code> . Sign JWT {id, role} with 7d expiry. Return {token, role, name}.
3	controllers/auth.controller.js — registerSC	Validate all fields. Check email not taken. <code>bcrypt.hash password</code> . Create User (status=pending). Create ServiceCentre. Send email to both admin emails: 'New SC registration pending approval'.
4	controllers/auth.controller.js — forgotPassword	Find user by email. Generate <code>Math.random</code> 6-digit code. Save OtpToken (10min expiry). <code>sendEmail(user.email, 'OTP for password reset', html with code)</code> .

#	File	What to build
5	controllers/auth.controller.js — verifyOtp	Find OtpToken by email+code where expiresAt > now. If found, mark as verified (or just return success — reset happens next step).
6	controllers/auth.controller.js — resetPassword	After OTP verified: bcrypt.hash new password, update User.passwordHash. Delete OtpToken.
7	middleware/auth.js	Extract 'Bearer token' from Authorization header. jwt.verify with JWT_SECRET. Attach req.user = {id, role}. 401 if missing or invalid.
8	middleware/rbac.js	isAdmin: (req,res,next) => req.user.role === 'admin' ? next() : 403. isSC: same for service_centre. isActive: check user status=active in DB.
9	routes/auth.routes.js	POST /auth/login, POST /auth/register, POST /auth/forgot-password, POST /auth/verify-otp, POST /auth/reset-password — all public
10	context/AuthContext.jsx	createContext. Store {user, token} in state. login(token, userData) saves to localStorage + state. logout() clears both. Wrap App.jsx in provider.
11	App.jsx	React Router v6. ProtectedRoute component: read token from context, redirect to /login if none. Role check: admin → /admin/*, sc → /sc/*. Default redirect by role after login.
12	pages/auth/Login.jsx	shadcn Form + Input (email, password) + Button. On submit call POST /auth/login. Store token via context.login(). Navigate by role.
13	pages/auth/Register.jsx	shadcn Form — all SC fields: ownerName, businessName, phone1, phone2, email1, email2, city (shadcn Select from /cities API), district (auto), state (auto), productCapability (3-option Select), password. On submit → POST /auth/register → show success message.
14	pages/auth/ForgotPassword.jsx	shadcn Form + Input (email). POST /auth/forgot-password. On success navigate to /verify-otp.
15	pages/auth/VerifyOtp.jsx	6 shadcn Inputs side by side (one digit each) or single Input. 10min countdown timer. POST /auth/verify-otp. Navigate to /reset-password.
16	pages/auth/ResetPassword.jsx	shadcn Form + 2 Inputs (new password, confirm). POST /auth/reset-password. Navigate to /login.

Test: register SC, check pending in Atlas. Login as admin (manually insert admin user). Test full OTP flow. Check Brevo dashboard to confirm email sent.

Phase 4 — Presets & Cities API (Day 3)

Goal: admin manages presets. City dropdown auto-fills district+state everywhere.

#	File	What to build
1	controllers/city.controller.js	getAll(): City.find().sort('name'). getByDistrict(district): filter. Used by city dropdowns everywhere.

#	File	What to build
2	routes/city.routes.js	GET /cities, GET /cities/district/:district — no auth (public, needed for register form too)
3	controllers/preset.controller.js	getAll(type?, isActive?): filter by query params. create(). update(id): name+price only. delete(id): check no complaints use it first. toggleActive(id).
4	routes/preset.routes.js	GET /presets (all — auth required), POST /presets (admin), PUT /presets/:id (admin), DELETE /presets/:id (admin), PATCH /presets/:id/toggle (admin)
5	components/forms/PresetSelector.jsx	shadcn Select. Fetches /presets?type=X&isActive=true. Shows name + ₹price. On select returns {presetId, presetPrice} to parent form.
6	pages/admin/Presets.jsx	shadcn Table of all presets grouped by type. shadcn Dialog for add/edit form. shadcn AlertDialog for delete confirm. Toggle switch (shadcn Switch) for active/inactive.

Phase 5 — Service Centre Management (Day 4-5)

Goal: admin approves/rejects SCs. Tab 2 complete. SC detail view working.

#	File	What to build
1	controllers/serviceCentre.controller.js	getAll(): filter by city/district/status/capability + search by name/phone. Pagination. getById(). approve(id): User.status=active, sendEmail SC approval. reject(id): status=rejected, sendEmail rejection. update(id): edit any SC field. deactivate(id). getStats(id): count complaints by status for this SC.
2	routes/serviceCentre.routes.js	GET /service-centres, GET /service-centres/pending, GET /service-centres/:id, PATCH /:id/approve, PATCH /:id/reject, PUT /:id, PATCH /:id/deactivate — all admin-only except /pending used in action centre
3	components/filters/SCFilters.jsx	Row of shadcn Selects: city, district, status, productCapability. shadcn Input for name/phone search (debounced 300ms).
4	pages/admin/ServiceCentres.jsx	SCFilters at top. shadcn Table or Card grid of SCs. Each row/card: businessName, city, productCapability Badge, stats (total/pending/done). Click → navigate to SCDetail.
5	pages/admin/SCDetail.jsx	shadcn Card with full SC info. Edit button → shadcn Dialog with editable form. Approve/Reject/Deactivate buttons (show based on current status). shadcn Tabs: Info tab, Complaints tab (reuse complaint list component).

Phase 6 — File Uploads (Day 5)

Goal: images and voice notes uploading to Cloudinary. Auto-compression active.

#	File	What to build
1	middleware/upload.js	multer-storage-cloudinary imageStorage: folder=microvison/images, transformation=[{width:1200,quality:80,crop:'limit'}], allowed jpg/png/webp, max 5MB. audioStorage: folder=microvison/audio, allowed webm/mp3/ogg/mp4, max 10MB.
2	controllers/upload.controller.js	uploadImages: req.files → return array of {url, public_id}. uploadAudio: req.file → return {url, public_id}.
3	routes/upload.routes.js	POST /upload/images (multer.array('images', 5), auth), POST /upload/audio (multer.single('audio'), auth)
4	components/forms/ImageUploader.jsx	Props: maxFiles (2 or 5), onUpload(urls). Drag+drop area using HTML drop events. On file select: POST /upload/images with FormData. Show thumbnail previews. Show progress. Show error if > maxFiles. Returns uploaded URLs to parent.
5	components/forms/VoiceRecorder.jsx	navigator.mediaDevices.getUserMedia({audio:true}). MediaRecorder start/stop. 60s max — setInterval timer, auto-stop at 60s. Collected chunks → Blob → POST /upload/audio as FormData. Show playback audio element after upload. Returns URL to parent.

iOS Safari fix: check MediaRecorder.isTypeSupported('audio/mp4') first, fallback to audio/webm. Always test voice recording on real mobile.

Phase 7 — Complaint Registration (Day 6-8)

Goal: admin registers a full complaint end-to-end. WhatsApp fires on submit. Reopen detection works.

#	File	What to build
1	utils/generateComplaintId.js	Complaint.findOne({createdAt: {\$gte: Jan1thisYear}}).sort({createdAt:-1}). Extract number from last ID, increment. Return MV-YYYY-XXXXX padded to 5 digits.
2	utils/reopenChecker.js	findEligible(phone1, product, complaintType): Complaint.findOne({phone1, product, complaintType, status:{\$in:['done','not_done']}, createdAt:{\$gte: 30daysAgo}). Returns complaint doc or null.
3	utils/sendWhatsApp.js	axios.post to WATI_BASE_URL/sendTemplateMessage. Body: {whatsappNumber: phone, template_name, broadcast_name, parameters:[]}. Accepts complaint object, builds params array.
4	controllers/complaint.controller.js — create	Validate. Run reopenChecker — if found return {reopenEligible: true, existingComplaint}. generateComplaintId. Snapshot presetPrice from Preset.findById. Create Complaint. Create ComplaintUpdate (status: new). Return complaint.

#	File	What to build
5	controllers/complaint.controller.js — assign	Find complaint. Find SC. Update complaint: assignedCentreId, assignedAt, status=assigned. Create ComplaintUpdate. Call sendWhatsApp with new_complaint template. Return updated complaint.
6	controllers/complaint.controller.js — reopenCheck	GET endpoint: call reopenChecker, return result.
7	routes/complaint.routes.js	POST /complaints, PATCH /complaints/:id/assign, GET /complaints/reopen-check — all admin-only
8	pages/admin/NewComplaint.jsx	Step wizard: useState for currentStep (1-4) + formData object. Step indicator at top (1→2→3→4 with shadcn pattern). Render current step component. Pass formData + setFormData as props. On step 4 submit: POST /complaints then PATCH /complaints/:id/assign.
9	components/forms/Step1CustomerInfo.jsx	shadcn Form. All customer inputs. On phone1 blur: GET /complaints/reopen-check → if hit, show ReopenBanner (shadcn Alert). Admin chooses: proceed to reopen flow or dismiss and continue new complaint.
10	components/forms/Step2ProductType.jsx	Product selection: 3 shadcn Cards (LED / Cooler / Both) acting as radio. Conditionally show type dropdown. Warranty: 2-option shadcn Select.
11	components/forms/Step3Charges.jsx	PresetSelector (in-warranty only, hidden for out-of-warranty). petrolAdmin shadcn Input. Extra charges: add-row button, each row has shadcn Input x2 (label + amount) + remove button. shadcn Textarea for notes. VoiceRecorder. ImageUploader (maxFiles=2).
12	components/forms/Step4AssignSC.jsx	Fetch /service-centres filtered by city+district+productCapability. shadcn Card for each SC showing businessName, city, load stats. Click to select (highlight selected). Submit button → triggers assign.
13	components/complaint/ReopenBanner.jsx	shadcn Alert with existing complaint summary. Two shadcn Buttons: 'Reopen this complaint' (opens Dialog for reopen notes + photos) or 'Create new complaint' (dismiss banner).

Phase 8 — Service Centre Portal (Day 9-11)

Goal: SC logs in, sees requests, accepts/rejects, updates status, uploads proof, fills charges.

#	File	What to build
1	controllers/complaint.controller.js — SC actions	getMyComplaints(scId, filters): complaints where assignedCentreId=scId. accept(id): status=accepted, log. reject(id): status=rejected_by_sc, log. markGoing(id): status=going, log.
2	controllers/complaint.controller.js — updateStatus	Validate SC owns complaint. If marking done: check proofPhotos.length > 0. Update status, proofPhotos,

#	File	What to build
		petrolSC (if edit 2 allowed), extraChargeRequest (push to extraCharges[] as pending), notes, customerPaymentAmount (out-of-warranty, required). Create ComplaintUpdate log.
3	routes/complaint.routes.js	GET /complaints/my (SC), PATCH /complaints/:id/accept (SC), PATCH /complaints/:id/reject (SC), PATCH /complaints/:id/going (SC), PATCH /complaints/:id/status (SC)
4	pages/sc/SCLayout.jsx	SCSidebar with 3 nav items + badge on New Requests (count from /complaints/my?status=assigned). Navbar top. React Router Outlet for pages.
5	pages/sc/NewRequests.jsx	GET /complaints/my?status=assigned. shadcn Card for each. Shows complaint ID, customer address+city, product, type, warranty Badge, preset info (if in-warranty), admin notes, VoiceNote player (HTML audio element), admin photos thumbnails. Two shadcn Buttons: Accept (shadcn Dialog confirm) / Reject (shadcn Dialog confirm).
6	pages/sc/MyComplaints.jsx	GET /complaints/my with all filters. ComplaintFilters at top (scoped). shadcn Table of complaints. Click row → open ComplaintDetail in Sheet (shadcn slide panel) or navigate to detail page.
7	components/complaint/ComplaintDetail.jsx — SC mode	Show all complaint info (read-only). SC action section at bottom based on current status: Mark Going button (if accepted). Proof upload (ImageUploader maxFiles=5, required before marking done). Status update form: shadcn Select (done/not_done/part_pending/replacement). PetrolEditField (if in-warranty + SC's turn). Extra charge request form (label + amount + shadcn Button to add). customerPaymentAmount shadcn Input (out-of-warranty only, required). Extra notes shadcn Textarea.
8	components/complaint/PetrolEditField.jsx	Props: petrolAdmin, petrolSC, petrolFinal, editCount, locked, userRole. Show current active value. Show history of all 3. If locked → show lock icon. If it's this role's turn (editCount determines whose turn) → show shadcn Input + save button.

Phase 9 — Admin Action Centre (Day 12-13)

Goal: Admin Tab 1 complete. All pending items visible and actionable.

#	File	What to build
1	controllers/complaint.controller.js — admin confirms	confirmDone(id): status=closed, billGenerated=true, billLockedAt=now, create ComplaintUpdate. disputeDone(id): status=accepted, create ComplaintUpdate with admin note.

#	File	What to build
		approveExtra(cld, extrald): find extraCharge in array, set status=approved. rejectExtra: set status=rejected.
2	controllers/complaint.controller.js — getActionItems	Return counts: pendingSCRegistrations (User.count where status=pending+role=sc), pendingConfirmations (Complaint.count where status=done), rejectedBySC (status=rejected_by_sc), pendingExtraApprovals (Complaint with extraCharges where status=pending, count total items).
3	routes/complaint.routes.js	PATCH /complaints/:id/confirm-done, PATCH /complaints/:id/dispute-done, PATCH /complaints/:id/extras/:extrald/approve, PATCH /complaints/:id/extras/:extrald/reject — all admin-only
4	pages/admin/ActionCentre.jsx	Fetch getActionItems for badge counts on AdminSidebar. 4 shadcn Card sections: (1) Pending SC Registrations — list with Approve/Reject shadcn Buttons + Dialog confirm. (2) Complaints marked done — list with Confirm (shadcn Button primary) and Dispute (shadcn Button variant) + Dialog for dispute note. (3) Rejected by SC — list with reassign button opens SC picker Dialog. (4) Extra charge requests — ExtraChargesList component with inline Approve/Reject.
5	components/complaint/ExtraChargesList.jsx	shadcn Table of extra charge line items. Columns: label, amount, requested by, status Badge. Admin sees Approve/Reject shadcn Buttons on pending rows. Approved/rejected show as locked Badge.
6	components/complaint/StatusTimeline.jsx	Vertical timeline. Map over ComplaintUpdate docs. Each item: colored dot (color by status), status label, actor name + role Badge, timestamp (relative + absolute on hover), note if any. Custom CSS — no shadcn equiv.

Phase 10 — Admin Complaints Tab (Day 14-15)

Goal: Admin Tab 3 — full complaint list, all filters, search, detail view with all admin actions.

#	File	What to build
1	controllers/complaint.controller.js — getAll	Support query params: q (search name/phone/complaintId), city, district, state, product, complaintType, warrantyStatus, status (comma-separated), assignedCentreId, isReopened, dateFrom, dateTo. Populate assignedCentre name. Sort by createdAt desc. Paginate 20/page.
2	routes/complaint.routes.js	GET /complaints — admin only. All query params passed through.

#	File	What to build
3	components/filters/ComplaintFilters.jsx	shadcn Input (search, debounced). Row of shadcn Selects: city, district, product, type, warranty, status (multi via checkboxes in DropdownMenu), SC name. Date range: two shadcn Inputs type=date. Reopened toggle. Clear all button.
4	pages/admin/AllComplaints.jsx	ComplaintFilters at top. shadcn Table: columns = ID, Customer, City, Product, Type, Warranty Badge, Status Badge, Assigned SC, Date. Row click → navigate to complaint detail or open shadcn Sheet. Pagination controls.
5	components/complaint/ComplaintDetail.jsx — admin mode	All info sections (read-only display). Admin action panel: Reassign SC (shadcn Button → Dialog with Step4AssignSC reused). PetrolEditField (admin's turn). ExtraChargesList (admin approve/reject). Confirm Done / Dispute Done buttons (if status=done). Reopen button (if eligible — check 30-day window). StatusTimeline at bottom. BillSummary (if status=closed).
6	hooks/useComplaints.js	useState for filters, page, complaints list. useEffect to fetch on filter change. Debounced search. Returns {complaints, loading, error, totalPages, setFilter, resetFilters}.

Phase 11 — Billing (Day 16-17)

Goal: billing calculations correct. Monthly invoice working. Admin Tab 4 and SC Tab 3 done.

#	File	What to build
1	utils/billingCalculator.js	calcBill(complaint): warrantyStatus=in_warranty → presetPrice + (petrolFinal petrolSC petrolAdmin 0) + sum(extraCharges where status=approved). out_of_warranty → sum(approved extras) only. Return {breakdown:{preset, petrol, extras:[]}, total, customerPaymentAmount}.
2	controllers/billing.controller.js	getComplaintBills(filters): Complaint.find({billGenerated:true, ...filters}). Map each through billingCalculator. Return with breakdown. Admin: any SC. SC: own only. getMonthlyInvoice(sclId, month, year): Complaint.find({assignedCentreId:sclId, billGenerated:true, month+year match}). Sum totals.
3	routes/billing.routes.js	GET /billing/complaints (role-scoped), GET /billing/invoice/:sclId (role-scoped)
4	components/billing/BillingTable.jsx	shadcn Table. Columns: Complaint ID, Customer, Product, Warranty Badge, Preset (₹), Petrol (₹), Extras (₹), Total (₹ bold), Customer Paid (₹ — out-of-warranty only, separate column). Footer row: totals.
5	components/billing/MonthlyInvoice.jsx	shadcn Card: SC name, month/year, complaint count, Microvision total owed (bold). Expandable (shadcn

#	File	What to build
		Collapsible) to show BillingTable of individual complaints.
6	components/complaint/BillSummary.jsx	shadcn Card inside ComplaintDetail (shown only when status=closed). Line items: Preset ₹X, Petrol ₹X, Extras ₹X each, Total ₹X bold. Out-of-warranty: shows extras only + 'Customer paid: ₹X' as separate note.
7	pages/admin/Billing.jsx	BillingFilters (month/year selects + SC select + all complaint filters). shadcn Tabs: 'Per Complaint' tab → BillingTable. 'Monthly Invoice' tab → list of MonthlyInvoice cards per SC.
8	pages/sc/SCBilling.jsx	Month+year shadcn Selects. MonthlyInvoice card (own data only). BillingTable below.
9	hooks/useBilling.js	Fetch billing data with filters. Returns {bills, invoices, loading, totals}.

Phase 12 — Reopen Flow (Day 18)

Goal: reopen detection + creation + WhatsApp trigger working end-to-end.

#	File	What to build
1	controllers/complaint.controller.js — reopen	Find original complaint. Validate eligibility (same phone1+product+complaintType, within 30 days, status=done/not_done). Create new Complaint: copy customer+product+warranty fields, isReopened=true, reopenParentId=originalId, reopenNotes, reopenPhotos, status=new, new MV-ID. Create ComplaintUpdate. sendWhatsApp with complaint_reopened template (include original ID in params).
2	routes/complaint.routes.js	POST /complaints/:id/reopen — admin only
3	components/complaint/ReopenBanner.jsx	shadcn Alert (yellow/warning) shows: original complaint ID, date, original status. shadcn Dialog for reopen form: Textarea for reopenNotes (required), ImageUploader maxFiles=2 (optional). Submit button POST /complaints/:id/reopen then navigate to new complaint.
4	pages/admin/NewComplaint.jsx — wire up	On reopen-check API hit: show ReopenBanner above step 1. If admin clicks 'Create new complaint': dismiss banner, continue form. If 'Reopen': open ReopenBanner Dialog, skip 4-step form.
5	components/complaint/ComplaintDetail.jsx — reopen button	Show 'Reopen Complaint' shadcn Button (outline, amber) when: role=admin AND status=done/not_done AND createdAt within last 30 days. Opens ReopenBanner Dialog inline.

Phase 13 — PWA + Polish + Deploy (Day 19-20)

Goal: PWA installable. Both repos live on Railway + Vercel. All production gotchas fixed.

#	File / Task	What to do
1	vite.config.js (finalize)	vite-plugin-pwa: registerType='autoUpdate', manifest={name:'Microvison',short_name:'MV',icons:[{src:'/icon-192.png',sizes:'192x192'},{src:'/icon-512.png',sizes:'512x512'}],theme_color:'#1E3A5F',background_color:'#ffffff',display:'standalone'}. Workbox: cache JS/CSS/HTML shell.
2	public/icon-192.png + icon-512.png	Create Microvison logo icons at both sizes. PNG format required for PWA.
3	components/ui/toaster.jsx in main.jsx	Add <Toaster /> to main.jsx so shadcn toasts work globally.
4	All pages — loading states	Wrap data fetches with shadcn Skeleton components. Show Skeleton table rows while loading. Use shadcn Alert for error states.
5	All pages — empty states	Custom EmptyState with icon + message when lists are empty (no complaints, no SCs, etc).
6	Railway deploy	Push backend to GitHub. Railway dashboard → New Project → Deploy from GitHub → select repo. Add all env vars one by one. Start command: node server.js. Set spending limit \$5 in Railway Billing settings. Get Railway URL.
7	MongoDB Atlas setup	Whitelist IP: 0.0.0.0/0 in Network Access (allows Railway). Get connection string. Add to Railway env as MONGO_URI.
8	Cloudinary setup	Create free account. Dashboard → API Keys. Add CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET to Railway.
9	Brevo setup	Create free account. Settings → API Keys → Generate. Add BREVO_API_KEY to Railway. Add + verify sender email (BREVO_FROM_EMAIL). Test: hit forgot-password API in production, confirm email arrives.
10	Vercel deploy	Push frontend to GitHub. Vercel dashboard → New Project → import repo. Add env var: VITE_API_URL = your Railway URL. Deploy. Test PWA: open on Chrome mobile → 'Add to Home Screen' appears.
11	WhatsApp provider setup	Create WATI or AiSensy account. Register message templates (new_complaint, complaint_reopened) — wait for Meta approval (24-48hrs). Add WATI_API_KEY + WATI_BASE_URL to Railway. Test both triggers from production.
12	Final checks	Test full flow: register SC → admin approves → admin registers complaint → WhatsApp fires → SC accepts → SC marks done → admin confirms → bill generated. Test OTP on mobile. Test PWA install on Android Chrome.

5. Approximate Lines of Code

Layer	Files	Avg lines/file	Total lines
Backend models	8	60	~480
Backend controllers	9	160	~1,440
Backend routes	9	35	~315
Backend middleware	4	50	~200
Backend utils + config	9	45	~405
server.js	1	60	~60
Backend subtotal	40	—	~2,900
Shadcn ui files (auto-generated)	19	—	~3,800 (not counted — not your code)
Frontend pages — auth (5)	5	90	~450
Frontend pages — admin (8)	8	200	~1,600
Frontend pages — sc (4)	4	140	~560
Frontend layout components (3)	3	80	~240
Frontend complaint components (7)	7	110	~770
Frontend form components (7)	7	140	~980
Frontend filter components (3)	3	70	~210
Frontend billing components (3)	3	90	~270
Frontend hooks + utils + context (8)	8	55	~440
Frontend config files (4)	4	40	~160
Frontend subtotal (hand-written)	57	—	~5,680
GRAND TOTAL (hand-written code)	97 files	—	~8,580 lines

Shadcn saves ~1,500-2,000 lines of UI code you would have written manually. Total project with shadcn files: ~12,000 lines. Hand-written by you: ~8,500.

6. All Environment Variables

6.1 Backend — set in Railway dashboard

Variable	Where to get it	Example
PORT	Set manually	5000
MONGO_URI	Atlas → Connect → Drivers	mongodb+srv://user:pass@cluster.mongodb.net/microvison?maxPoolSize=5
JWT_SECRET	Generate any 64-char random string	supersecretlongstring123...
CLOUDINARY_CLOUD_NAME	Cloudinary dashboard	your-cloud-name
CLOUDINARY_API_KEY	Cloudinary dashboard → API Keys	123456789012345
CLOUDINARY_API_SECRET	Cloudinary dashboard → API Keys	AbCdEfGhIjKlMnOp
BREVO_API_KEY	Brevo → Settings → API Keys	xkeysib-abc123...
BREVO_FROM_EMAIL	Verified sender in Brevo	noreply@microvison.in
WATI_API_KEY	WATI dashboard → API	eyJhbGciOi...
WATI_BASE_URL	WATI dashboard	https://live-server-XXXX.wati.io
FRONTEND_URL	Your Vercel deploy URL	https://microvison.vercel.app

6.2 Frontend — set in Vercel dashboard

Variable	Value
VITE_API_URL	https://your-app-name.railway.app

7. Production Gotchas — Things That Break Only in Production

Issue	Why	Fix
SMTP / Nodemailer fails on Railway	Railway Hobby blocks all SMTP ports — confirmed on official pricing page	Use Brevo HTTP API in <code>sendEmail.js</code> . Never use nodemailer SMTP on Railway Hobby.
CORS error in production	Backend doesn't allow Vercel domain	<code>cors({origin: process.env.FRONTEND_URL})</code> in <code>server.js</code> . Set <code>FRONTEND_URL</code> in Railway env.
MongoDB drops connections	Atlas M0 has low connection pool limit	Add <code>?maxPoolSize=5</code> to <code>MONGO_URI</code> . Don't open new connections per request.
Cloudinary fails silently	Missing env vars on Railway	Verify all 3 Cloudinary vars in Railway dashboard. Check deploy logs for config errors.
JWT works local, fails production	Wrong <code>JWT_SECRET</code> or missing env var	Copy-paste exact value. Check Railway env var has no trailing space.
Images not compressing	Cloudinary transformation not in upload middleware	Add <code>transformation:[{width:1200,quality:80,crop:'limit'}]</code> to <code>multer-storage-cloudinary</code> config.
PWA not updating after redeploy	Old service worker serving cached files	<code>vite-plugin-pwa</code> <code>registerType:'autoUpdate'</code> handles this automatically.
Voice note fails on iOS Safari	Safari doesn't support <code>audio/webm</code> — different codec	In <code>VoiceRecorder.jsx</code> : check <code>MediaRecorder.isTypeSupported('audio/mp4')</code> first, use as <code>mimeType</code> if supported.
WhatsApp templates rejected	Template not pre-approved by Meta before going live	Submit templates in WATI/AiSensy dashboard and wait 24-48hrs for Meta approval before testing production.
Railway charges over \$5	Multiple services running inside Railway project	Keep exactly 1 service in Railway — your Node.js app. DB on Atlas, files on Cloudinary. Set \$5 spend limit in Railway Billing.
Shadcn components not found	Import path wrong after shadcn init	Shadcn components are in <code>src/components/ui/</code> . Import as: <code>import { Button } from '@components/ui/button'</code> . Configure <code>@</code> alias in <code>vite.config.js</code> .

8. Realistic Timeline

Day	Phase	Done by end of day
1	Phase 1 — Setup	Both repos running, shadcn initialized, all 19 components added, DB connected
1-2	Phase 2 — Models	All 8 schemas, cities seeded, verified in Atlas
2-3	Phase 3 — Auth	Login, register, OTP reset working — tested in Postman + browser. Brevo email confirmed.
3	Phase 4 — Presets & Cities	Preset CRUD done, city dropdown auto-fills district+state
4-5	Phase 5 — SC Management	Admin approves/rejects SCs, Tab 2 UI done
5	Phase 6 — Uploads	Images + voice notes uploading to Cloudinary with compression
6-8	Phase 7 — Complaint Registration	Full 4-step form, WhatsApp fires on submit
9-11	Phase 8 — SC Portal	SC accepts, updates status, uploads proof, fills charges
12-13	Phase 9 — Action Centre	Admin confirms done, approves extras, Tab 1 done
14-15	Phase 10 — Complaints Tab	All filters + search + admin detail actions working
16-17	Phase 11 — Billing	Bills correct, invoices working, Tab 4 and SC Tab 3 done
18	Phase 12 — Reopen	Reopen detection + creation + second WhatsApp trigger done
19-20	Phase 13 — PWA + Deploy	Live on Railway + Vercel. PWA installable on mobile + laptop.

20 working days at vibe coding pace (AI-assisted). Shadcn saves roughly 3-4 days of frontend work vs writing raw Tailwind from scratch.